

Data Science Roadmap (Revised) — Zero to Job-Ready

Core philosophy (keep this pinned above your desk):

Every major topic ends with a project. You're not "finishing a course" — you're building a portfolio that proves you can take raw data to insights, predictions, and a decision.

Timeline note: The original plan compresses ~15 distinct ML topics into 8 weeks. That's exposure, not mastery. This version runs ~**24 weeks** (~5.5 months) assuming 25–30 focused hours/week. If you're full-time job hunting or studying 40+ hrs/week, you can compress it back down — but don't compress the *review* weeks. They're what makes this stick.

Phase 0: Environment Setup (2–3 Days)

- Install: Python (3.11+), VS Code, Jupyter, Git, Anaconda/Miniconda
 - Create GitHub account + set up SSH keys
 - Learn: virtual environments (`venv` or `conda`), package installation, basic Git (`clone`, `add`, `commit`, `push`, `branch`, `pull request`)
 - **New:** Set up a single GitHub repo now called `data-science-portfolio` — every project from Phase 1 onward lives here in its own folder. Don't wait until Phase 9 to organize your portfolio; organize as you go.
-

Phase 1: Python for Data Science (Weeks 1–2)

NumPy — arrays, indexing, broadcasting, vectorization, math ops, random module

Pandas — Series, DataFrames, reading CSV/Excel/JSON, filtering, sorting, GroupBy, merge/join, pivot tables, `.apply()`, missing values, datetime operations

Jupyter — Markdown, documentation habits, notebook organization (this matters more than people think — a messy notebook reads as messy thinking to an interviewer)

Mini Projects (2–3, no ML yet):

- Netflix Movies, FIFA Players, or Spotify Songs datasets
- Answer: most common genre? which country produces the most content? trends over time?

Checkpoint: Can you load a messy CSV, clean it, and answer 3 business questions without looking anything up? If not, stay here one more day before moving on.

Phase 2: SQL Review (Week 3)

Since you've already studied SQL, this is a refresh + gap-fill, not a first pass.

- **Basic:** SELECT, WHERE, ORDER BY, LIMIT
- **Intermediate:** GROUP BY, HAVING, CASE, aggregate functions
- **Advanced:** JOINS (all types), CTEs, window functions (`ROW_NUMBER`), (`RANK`), (`LAG`)/(`LEAD`), subqueries

Practice 30–50 problems on LeetCode SQL or HackerRank SQL, weighted toward window functions — they come up constantly in interviews and most people are weak on them.

Project: Analyze a sales database (e.g., Chinook or Northwind) and answer business questions — top customers, monthly revenue trend, cohort-style repeat purchase rate.

Phase 3: Exploratory Data Analysis (Weeks 4–6)

This is the most important phase in the whole roadmap — most of your actual job will look like this.

Data Cleaning

- Missing values, duplicate rows, outliers, incorrect dtypes, feature engineering basics

Statistics — learned in context, not as a standalone math unit

- Mean vs. median, standard deviation, correlation, distribution shape, skewness, percentiles, IQR, z-score, confidence intervals (when applicable)

Visualization

- Matplotlib: line, bar, histogram, scatter, box plot, heatmap
- Seaborn for statistical plots (recommended, not optional — it's the industry default for quick EDA)

Projects: Build two complete EDA notebooks. Each must include:

1. Problem statement (what decision would this analysis inform?)
2. Cleaning steps, documented
3. Visualizations with a one-line "so what" under each
4. Business insights

5. Conclusions

Push both to GitHub with a proper README.

New — Communicating findings (half a day): Write a 1-page, non-technical summary of one EDA project as if for a manager who won't open the notebook. This single habit disproportionately separates "can code" candidates from "can be trusted with a stakeholder" candidates.

Phase 4: Power BI (Weeks 7-8)

- **Data Import:** Excel, CSV, SQL
- **Power Query:** cleaning, transformations
- **Data Modeling:** relationships, star schema basics
- **DAX:** SUM, CALCULATE, FILTER, time intelligence functions
- **Dashboards:** Sales, HR, Financial — 3 dashboards, each telling one clear story

Skill to add: for each dashboard, write 3 bullet "insights" a VP could read in 10 seconds. Dashboards without a narrative are just charts.

Review Week (Week 9)

Not in the original plan — added deliberately. Revisit Phase 1-3 material:

- Redo one NumPy/Pandas exercise from memory
- Re-derive z-score and IQR outlier detection without notes
- Skim your own EDA notebooks — could you explain every cell to a stranger?

This is cheap insurance against the "forgot everything by week 18" problem.

Phase 5: Machine Learning (Weeks 10-19)

The longest and most important phase. Given a full week per major topic instead of cramming.

Week 10 — Regression

- Linear, Multiple Regression
- Evaluation: MAE, MSE, RMSE, R^2
- Diagnostics: residuals, linearity, multicollinearity (VIF)

- **Project:** House Price Prediction

Week 11 — Classification

- Logistic Regression, KNN
- Evaluation: accuracy, precision, recall, F1, ROC-AUC — and *when each metric matters* (e.g., recall over accuracy for fraud/churn)
- **Project:** Customer Churn

Week 12 — Tree-Based Models

- Decision Trees, Random Forest
- Entropy, Gini, feature importance
- **Project:** Loan Prediction

Week 13 — Advanced Classical Models

- SVM, Naive Bayes
- Bayes' theorem, kernel intuition (not the full math)
- **Project:** Email/SMS Spam Classifier (Naive Bayes is the textbook fit here, and it's a clean, fast way to compare against your Week 11 Logistic Regression churn model on the same evaluation metrics)

Week 14 — Unsupervised Learning

- K-Means, Hierarchical Clustering
- **Project:** Customer Segmentation

Week 15 — Dimensionality Reduction

- PCA
- **Project:** Visualize a high-dimensional dataset in 2D

Week 16 — Model Improvement

- Cross-validation, Grid Search, Random Search, Pipelines

Week 17 — Feature Engineering

- Scaling, encoding, feature selection
- **Capstone:** one end-to-end project combining everything above

Week 18 — Light NLP (added)

- Bag-of-words, TF-IDF, basic sentiment classification with scikit-learn (no deep learning needed)
- **Project:** Sentiment classifier on product reviews or tweets
- *Why this matters:* text data comes up constantly in real DS roles, and "I've never touched text" is a noticeable gap even for classical-ML-focused positions.

Week 19 — Light Time Series (added)

- Trend/seasonality decomposition, moving averages, basic forecasting (ARIMA or Prophet at an intuition level)
 - **Project:** Forecast next month's sales from historical data
 - Optional 1-day add-on: what an A/B test is, how to read a p-value, why sample size matters — increasingly asked about even in junior interviews, especially product-adjacent roles.
-

Phase 6: Git & GitHub (Ongoing, Throughout)

Every project, no exceptions:

- README (problem, approach, results, how to run it)
 - `requirements.txt`
 - Clear folder structure
 - Real commit history (not one giant "final commit") — interviewers do look at this
-

Phase 7: Kaggle (Start Week 14, alongside ML phase)

Don't chase leaderboard rank. Use it to see how experienced people think.

- Start with Titanic, then House Prices
 - Read 3-5 winning/high-voted notebooks per competition
 - Note *why* they made the choices they did, not just what they did
-

Phase 8: Deployment (Weeks 20-21)

- **Streamlit:** deploy Customer Churn Predictor and House Price Predictor as interactive apps

- **Optional but recommended:** FastAPI for a proper backend + Docker basics — even a shallow "I containerized this" is a strong signal in interviews
-

Phase 9: Portfolio Consolidation (Weeks 22–23)

Your GitHub should now contain:

- **Data Analysis:** Netflix Analysis, Sales Analysis
- **Dashboards:** Power BI Sales Dashboard, HR Dashboard
- **Machine Learning:** House Prices, Customer Churn, Loan Prediction, Customer Segmentation
- **NLP:** Sentiment Classifier
- **Time Series:** Sales Forecast
- **Deployment:** Live Streamlit app(s), linked and working (not just code — an actual clickable link)
- Optional: a recommendation system, if you have appetite

New — polish pass: rewrite every README's first paragraph as "the problem, the approach, the result" in 3 sentences. Recruiters skim; the first paragraph is what gets read.

Phase 10: Interview Preparation (Week 24+, ongoing)

Python: lists, dictionaries, OOP basics, NumPy, Pandas

SQL: medium JOINS, window functions

Machine Learning — be able to explain, not just do:

- Why Random Forest instead of Logistic Regression?
- What is overfitting, and how do you detect it?
- What is cross-validation, and why does it matter?
- What is feature engineering, and give an example from your own project?
- Why did you choose this evaluation metric over another?
- **New:** How would you explain [your model] to a non-technical stakeholder?

Behavioral: for every project — the problem, your approach, challenges you hit, the result, and what you'd do differently now. This last part (what you'd change) is underused and signals real seniority even at junior level.

Resources

- **Python:** official docs, Kaggle Learn
 - **SQL:** LeetCode SQL, HackerRank SQL
 - **Machine Learning:** *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (Aurélien Géron), scikit-learn docs
 - **Visualization:** Matplotlib docs, Seaborn gallery
 - **Power BI:** Microsoft Learn, practical dashboard tutorials
 - **NLP/Time Series (new):** scikit-learn text tutorial, `statsmodels` docs, Prophet quickstart
-

Video-Based Resources by Phase

Direct links to specific videos or playlists where I could verify the exact URL; where a series is well known but I couldn't pin an exact playlist link, I've linked the creator's video list with the search term to use (it'll be the first result).

Phase 0 — Environment Setup

- [Git & GitHub Crash Course for Beginners — freeCodeCamp](#)

Phase 1 — Python for Data Science (NumPy, Pandas, Jupyter)

- NumPy:
 - [Python NumPy Tutorial for Beginners — Keith Galli \(via freeCodeCamp\)](#) — arrays, indexing/slicing, broadcasting, math, linear algebra, statistics, reshaping; a full 1-hour course covering everything the roadmap lists
- Pandas:
 - [Python Pandas Tutorial \(full playlist\) — Corey Schafer](#) — DataFrames, filtering, GroupBy, cleaning, dates, reading/writing CSV/Excel/JSON/SQL, part by part
 - [Keith Galli — Videos](#) — search "Complete Python Pandas Data Science Tutorial" and "Solving Real-World Data Science Tasks with Pandas"
 - [Data School — Videos](#) — search "Easier data analysis with pandas" if Corey Schafer's pace feels too slow
- Jupyter Notebook (markdown, documentation, organization):
 - [Corey Schafer — Videos](#) — search "Jupyter Notebook Tutorial" for setup, markdown cells, and keyboard-shortcut workflow basics

Phase 2 — SQL Review

- [SQL Tutorial for Beginners \(full playlist\)](#) — [Alex The Analyst](#) — basic through advanced, including JOINS, CTEs, and window functions
- Practice on [LeetCode SQL 50](#) and [HackerRank SQL](#) — SQL is a "do it" skill, videos alone won't get you there

Phase 3 — Exploratory Data Analysis, Statistics & Visualization

- Statistics:
 - [Statistics Fundamentals \(full playlist\)](#) — [StatQuest with Josh Starmer](#) — mean/median/variance, distributions, hypothesis testing, p-values, confidence intervals
- EDA / data cleaning:
 - [Luke Barousse — Videos](#) — search "Python for Data Analytics" full course
 - [Krish Naik — Videos](#) — search his EDA project walkthroughs
- Matplotlib:
 - [Matplotlib Tutorial \(full playlist\)](#) — [Corey Schafer](#) — line, bar, pie, histogram, scatter, and subplot chapters, one topic per video
- Seaborn:
 - [Seaborn Crash Course — Data Visualization in Python](#) — box plots, heatmaps, and the statistical plot types the roadmap calls for, built on top of Matplotlib

Phase 4 — Power BI

- [Power BI Tutorial for Beginners \(Step-by-Step in 30 Minutes\)](#) — [Guy in a Cube](#)
- [Curbal — full channel](#) — search "DAX Fridays" for the weekly DAX series
- [SQLBI — Videos](#) (Marco Russo & Alberto Ferrari) — deepest source on DAX and data modeling once basics are solid

Phase 5 — Machine Learning

- [A Gentle Introduction to Machine Learning](#) — [StatQuest](#), then browse [StatQuest's playlists page](#) for Linear/Logistic Regression, Decision Trees, Random Forests, SVM, Naive Bayes, K-Means, and PCA — do these *before* the scikit-learn implementation videos so the code isn't a black box
- [Machine Learning Playlist \(100+ videos\)](#) — [Krish Naik](#) — end-to-end pipelines, regression through tree-based models
- [Data School — Videos](#) — search "Machine Learning in Python with scikit-learn"
- [Essence of Linear Algebra \(full playlist\)](#) — [3Blue1Brown](#) — optional, but useful for understanding what regression and PCA are doing under the hood

- Same [freeCodeCamp Git crash course](#) from Phase 0; revisit the branching/merge-conflict timestamps once you're managing multiple project folders

Phase 7 — Kaggle

- [Kaggle Learn](#) — free micro-courses built around Kaggle's own datasets and competitions
- [Ken Jee — Videos](#) — search "How I Would Learn Data Science" and his Kaggle/portfolio-review videos

Phase 8 — Deployment (Streamlit)

- [Building a Machine Learning App in Python with Streamlit](#) — Data Professor
- [365 Data Science — Videos](#) — search "Deploying Machine Learning Models with Streamlit"
- Optional (FastAPI + Docker): [Deploy Machine Learning Models with FastAPI & Docker — Full Tutorial](#) — takes a trained model from notebook to a containerized, callable API

Phase 5 add-ons — Light NLP & Time Series

- NLP: [Krish Naik — Videos](#) — search his NLP playlist for TF-IDF/bag-of-words/sentiment classification (skip the deep-learning NLP videos — out of scope per the "what to skip" list)
- Time series: [freeCodeCamp — Videos](#) — search "time series forecasting" for trend/seasonality decomposition and ARIMA basics

Phase 9 — Portfolio

- [Ken Jee — Videos](#) — search his portfolio and resume-review videos

Phase 10 — Interview Prep

- [StatQuest playlists](#) — for re-explaining any ML concept you can't articulate cleanly
- [Krish Naik — Videos](#) — search his data science interview question playlists

How to use this without drowning in tutorials: pick exactly one channel per skill area (Python → Corey Schafer or Keith Galli; SQL → Alex The Analyst; stats/ML theory → StatQuest; BI → Guy in a Cube) and stick with it end to end. Switching channels mid-topic is the #1 way people stall out — a slightly-less-perfect channel you finish beats a "better" one you keep restarting.

What to Deliberately Skip (for now)

- ✗ R
 - ✗ Tableau (until a job specifically needs it)
 - ✗ TensorFlow/PyTorch (until classical ML is genuinely solid)
 - ✗ Deep mathematical proofs
 - ✗ Big Data tools (Spark, Hadoop) unless a project or job calls for them
-

Summary of What Changed vs. the Original Plan

Change	Why
18 weeks → 24 weeks	Original ML phase (8 weeks for 9+ major topics) was mastery-in-name-only
Added a Week 9 review checkpoint	Prevents front-loaded material from decaying by week 18
Added light NLP (Week 18)	Text data is common in real DS work; total gap here is a red flag
Added light time series + A/B testing intuition (Week 19)	Forecasting and experimentation show up constantly in junior interviews
Added stakeholder-communication exercises in Phases 3 & 9	"Can code" vs. "can be trusted with a business question" is the real hiring bar
Moved portfolio organization to Phase 0	Building the habit early beats a scramble at Week 17
Added "what would you do differently" to behavioral prep	Signals judgment, not just execution

Bottom line: the original plan's bones were excellent — project-first, right sequencing, smart exclusions. This version keeps all of that and fixes the two real gaps: unrealistic time compression, and a portfolio that was 100% technical with no evidence you can talk to a non-technical audience.